

CineRate Project Report

Technical Overview & Application Logic

1. Project Executive Summary

CineRate is a modern Single Page Application (SPA) designed for movie enthusiasts. It functions as a dynamic discovery tool and a personal review diary. By integrating external Real-Time data with local browser persistence, it provides a seamless user experience for tracking watched films.

HTML5

CSS3 Variables

Vanilla JavaScript

TMDB API

Local Storage

2. How the Code Works Together

The application follows a circular data flow logic, moving from external API requests to local user interactions.

A. The Data Fetching Cycle (External)

The application uses `async/await` syntax to handle asynchronous communication with The Movie Database (TMDB). When the page loads or a search is performed:

- The script constructs a URL containing the API key and query parameters.
- The `fetch()` command sends this to TMDB.
- The `displayMovies()` function takes the resulting JSON and maps it into HTML elements, injecting them into the DOM.

B. State Management & Modals

When a user clicks a movie card, the application enters a "Focus State":

- The `currentMovieId` variable is updated to match the selected movie.
- The Modal's CSS is changed from `display: none` to `display: block`.

- The `loadReviews()` function triggers a lookup in the Local Storage using the unique Movie ID.

3. Component Interaction Matrix

Component	Responsibility	Connection Method
Search Engine	Filters TMDB results based on user input.	Event Listener (Submit)
Grid System	Responsive display of movie posters.	CSS Grid / DOM Injection
Local Vault	Stores reviews across browser sessions.	Web Storage API (localStorage)
UI Modal	Input interface for user ratings.	Global State Management

4. Data Persistence Logic

Because the app doesn't use a backend database, it treats the browser's `localStorage` as a "key-value" store. The code handles data transformation as follows:

The JSON Workflow:

1. **Storage:** Data is converted to a string using `JSON.stringify()`.
2. **Retrieval:** Strings are converted back to usable JavaScript objects using `JSON.parse()`.

5. Design & User Experience (UX)

The design utilizes a **Dark UI** pattern to mimic the cinema environment. Key features include:

- **Responsive Breakpoints:** The movie grid adapts from 1 column on mobile to 5+ columns on desktop.
- **Visual Feedback:** Card elevation (`translateY`) on hover to signify interactability.
- **Input Validation:** Logic to prevent empty reviews or missing API key errors.